DLCD-count16_7seg-vhdl-assignment

Name - 최원창 (Choi Won Chang)

Student ID - 2023032615

Major - Robotics Engineering

1. Objective

Implement a VHDL module that counts the ones in the 16-bit input vt and displays counts 0 to 15 as 0 to F. The enable input en controls whether the seven-segment display is on. For count 16, this design blanks the display as an additional design choice.

16 비트 입력 vt 의 1 의 개수를 세어 0~15 를 0~F 로 표시하고, en 으로 표시 여부를 제어하는 VHDL 모듈을 구현합니다. 16 개 모두 1 인 경우는 추가 설계 방침으로 전체 소등합니다.

2. VHDL Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity count16_7seg is
    port (
        vt : in STD_LOGIC_VECTOR(15 downto 0);
        en : in STD_LOGIC;
        sa, sb, sc, sd, se, sf, sg : out STD_LOGIC
    );
end count16_7seg;
architecture Behavioral of count16_7seg is
    -- Segment order: a b c d e f g
    -- '1' = ON, '0' = OFF
    signal segments : STD_LOGIC_VECTOR(6 downto 0);
begin
    process(vt, en)
        variable count : integer range 0 to 16;
    begin
        count := 0;
        for i in vt'range loop
            if vt(i) = '1' then
                count := count + 1;
            end if;
        end loop;
        if en = '1' then
            case count is
                when 0  => segments <= "1111110"; -- 0
                when 1  => segments <= "0110000"; -- 1
                when 2  => segments <= "1101101"; -- 2
                when 3  => segments <= "1111001"; -- 3
                when 4  => segments <= "0110011"; -- 4
                when 5  => segments <= "1011011"; -- 5
                when 6  => segments <= "1011111"; -- 6
                when 7  => segments <= "1110000"; -- 7
                when 8  => segments <= "1111111"; -- 8
                when 9  => segments <= "1111011"; -- 9
                when 10 => segments <= "1110111"; -- A
                when 11 => segments <= "0011111"; -- b
                when 12 => segments <= "1001110"; -- C
                when 13 => segments <= "0111101"; -- d
                when 14 => segments <= "1001111"; -- E
                when 15 => segments <= "1000111"; -- F
                when others => segments <= "0000000";
            end case;
        else
            segments <= "0000000";
        end if;
    end process;
    sa <= segments(6);
    sb <= segments(5);
    sc <= segments(4);
    sd <= segments(3);
    se <= segments(2);
    sf <= segments(1);
    sg <= segments(0);
end Behavioral;
```

3. Circuit Design

The entity count16_7seg uses vt(15 downto 0), a one-bit enable input en, and outputs sa through sg. The process sets count to 0 and counts every input bit equal to 1. It then selects a seven-segment pattern. This is a combinational population count, with no clock or accumulation over time.

count16_7seg 는 입력 vt(15 downto 0), 1 비트 en, 출력 sa~sg 를 사용합니다. 프로세스에서 count 를 0 으로 초기화하고 1 인 입력 비트를 모두 센 뒤 표시 패턴을 선택합니다. 클록이나 시간에 따른 누적 기능이 없는 조합논리 방식의 비트 개수 계산 회로입니다.

The segment order is A B C D E F G, with 1 meaning on and 0 meaning off. Outputs sa through sg connect to tunnels A through G on the display. When en = 0, all seven outputs are 0. B and D use the familiar lowercase b and d shapes to distinguish them from 8 and 0.

세그먼트 순서는 A B C D E F G 이며 1 은 점등, 0 은 소등입니다. sa~sg 는 표시기의 터널 A~G 에 연결합니다. en = 0 이면 모든 출력이 0 입니다. B 와 D 는 8 및 0 과 구분하기 위해 소문자 b 와 d 형태로 표시합니다.

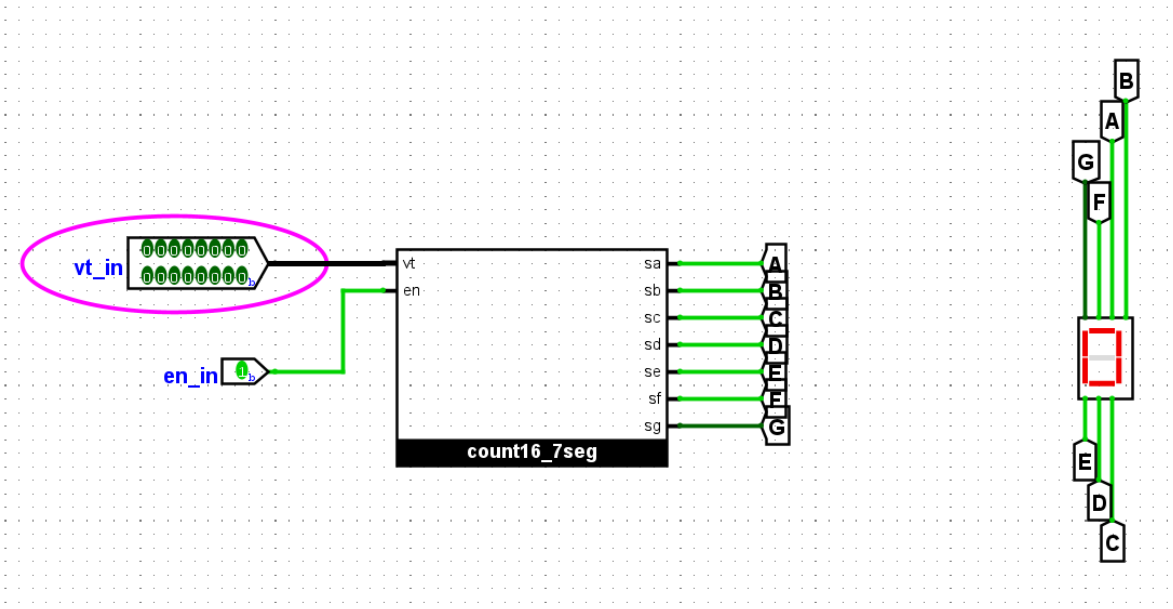


Figure 1. The count16_7seg circuit displaying 0 for an all-zero input.

3.1 Handling a Count of 16

A 16-bit input can contain 16 ones, while a single hexadecimal digit represents only 0 to 15. The assignment does not specify a display for 16. This implementation uses when others to output 0000000, so vt = 1111111111111111 with en = 1 deliberately turns all segments off. It is not displayed as 0.

16 비트 입력에는 1 이 최대 16 개 존재하지만, 한 자리 16 진 표시는 0~15 까지만 표현합니다. 과제에서 16 의 표시 방법을 지정하지 않았으므로 when others 에서 0000000 을 출력하도록 정했습니다. 따라서 vt 가 모두 1 이고 en = 1 인 경우 전체 소등하며, 숫자 0 을 표시하는 것과 구분됩니다.

4. Simulation Results

The 17 supplied screenshots cover every possible count from 0 to 16 with en = 1. The data values below are written from vt(15) on the left to vt(0) on the right. Each observed display matches the specified segment pattern.

제공된 17 개 스크린샷은 en = 1 에서 가능한 모든 개수 0~16 을 포함합니다. 아래 데이터는 왼쪽 vt(15)부터 오른쪽 vt(0) 순서입니다. 각 화면의 표시 결과가 정의된 세그먼트 패턴과 일치합니다.

Ones	vt(15 downto 0)	A B C D E F G	Display	Observed
0	00000000 00000000	1111110	0	Matches
1	00000000 00000001	0110000	1	Matches
2	00000000 00000011	1101101	2	Matches
3	00100000 00000011	1111001	3	Matches
4	00100000 10000011	0110011	4	Matches
5	10100000 10000011	1011011	5	Matches
6	10100000 10001011	1011111	6	Matches
7	10100100 10001011	1110000	7	Matches
8	11100100 10001011	1111111	8	Matches
9	11100101 10001011	1111011	9	Matches
10	11100101 10101011	1110111	A	Matches
11	11100101 11101011	0011111	b	Matches
12	11100101 11111011	1001110	C	Matches
13	11110101 11111011	0111101	d	Matches
14	11110101 11111111	1001111	E	Matches
15	11110111 11111111	1000111	F	Matches
16	11111111 11111111	0000000	Blank	Matches

Enable-off behavior: for en = 0, the source assigns 0000000 regardless of vt. The supplied screenshots all show en = 1, so no enable-off observation is claimed here. Counts 0 to 15 follow the required mapping; blanking at 16 is the additional behavior described above.

en = 0 이면 vt 와 관계없이 0000000 을 출력하도록 코드에 정의되어 있습니다. 제공된 캡처는 모두 en = 1 이므로 en = 0 의 화면 검증 결과는 포함하지 않았습니다. 0~15 는 요구된 표시를 따르며, 16 의 소등은 앞서 설명한 추가 동작입니다.

4.1 Case 1 Count = 0

vt = 00000000 00000000; en = 1; pattern A–G = 1111110.

1 의 개수는 0 개이며, 관측한 표시 0 가 예상 결과와 일치합니다.

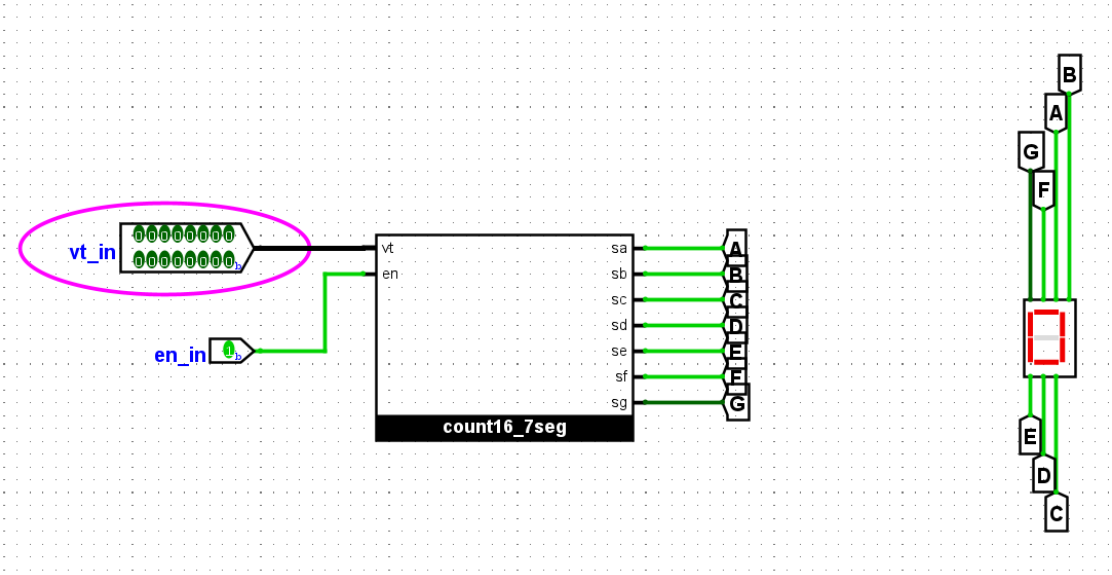


Figure 2. Count = 0, en = 1; observed display: 0.

4.2 Case 2 Count = 1

vt = 00000000 00000001; en = 1; pattern A–G = 0110000.

1 의 개수는 1 개이며, 관측한 표시 1 가 예상 결과와 일치합니다.

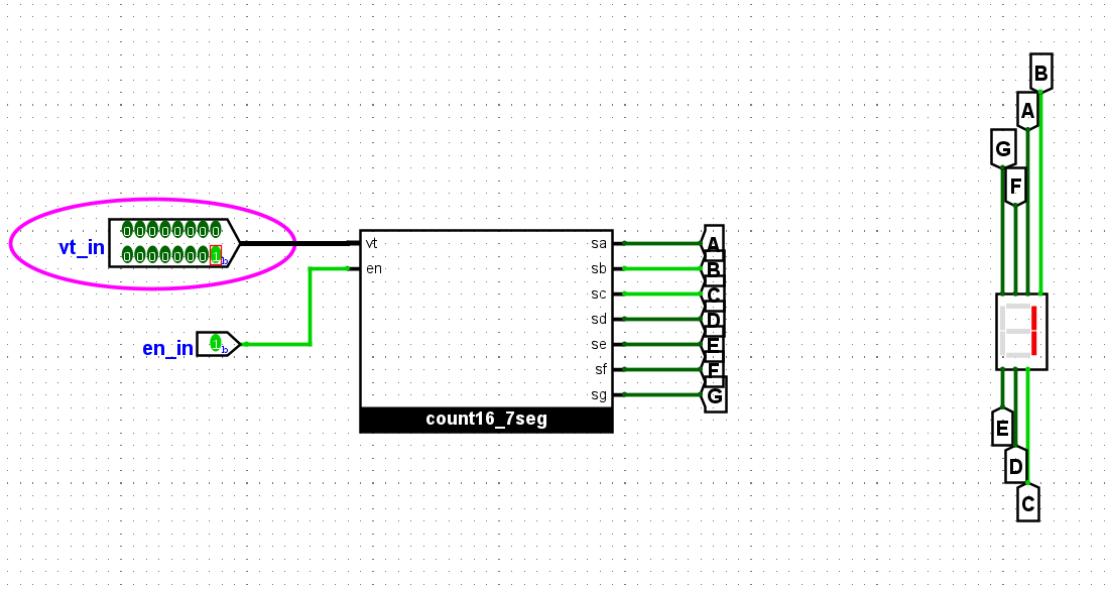


Figure 3. Count = 1, en = 1; observed display: 1.

4.3 Case 3 Count = 2

vt = 00000000 00000011; en = 1; pattern A–G = 1101101.

1 의 개수는 2 개이며, 관측한 표시 2 가 예상 결과와 일치합니다.

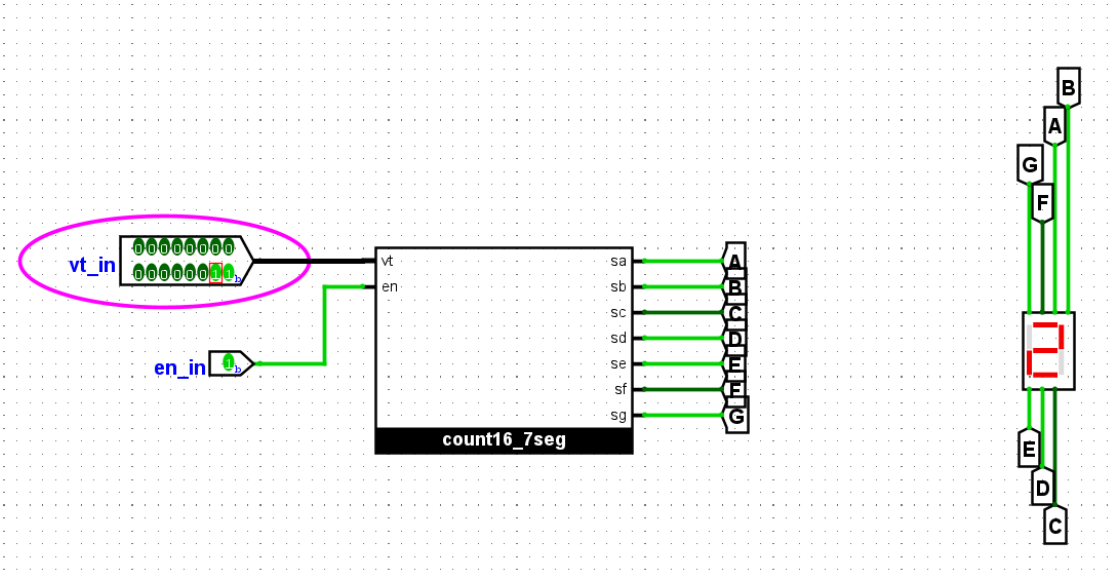


Figure 4. Count = 2, en = 1; observed display: 2.

4.4 Case 4 Count = 3

vt = 00100000 00000011; en = 1; pattern A–G = 1111001.

1 의 개수는 3 개이며, 관측한 표시 3 가 예상 결과와 일치합니다.

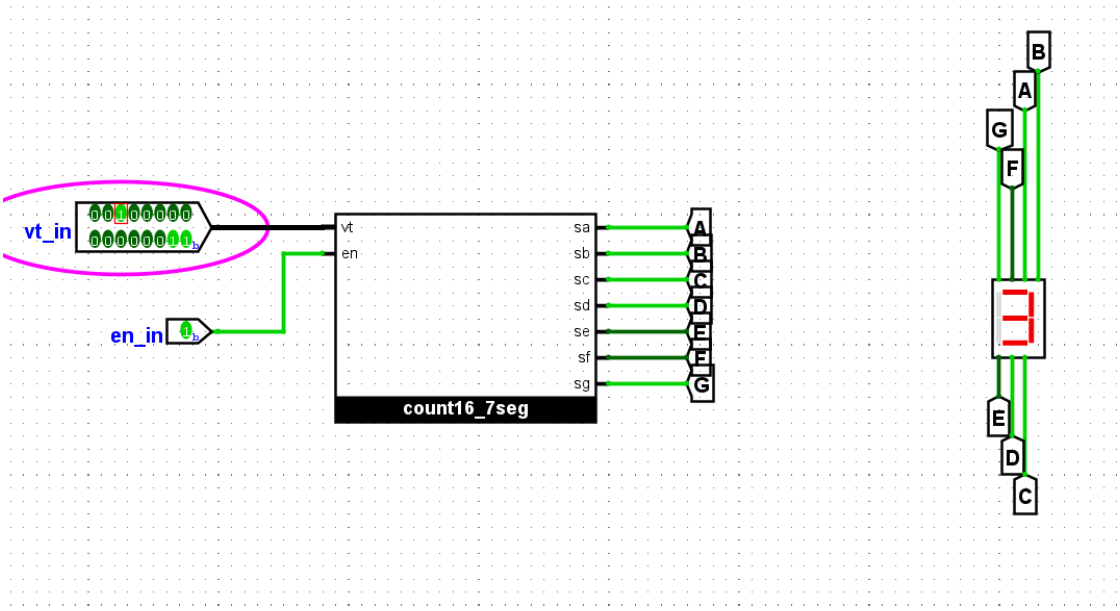


Figure 5. Count = 3, en = 1; observed display: 3.

4.5 Case 5 Count = 4

vt = 00100000 10000011; en = 1; pattern A–G = 0110011.

1 의 개수는 4 개이며, 관측한 표시 4 가 예상 결과와 일치합니다.

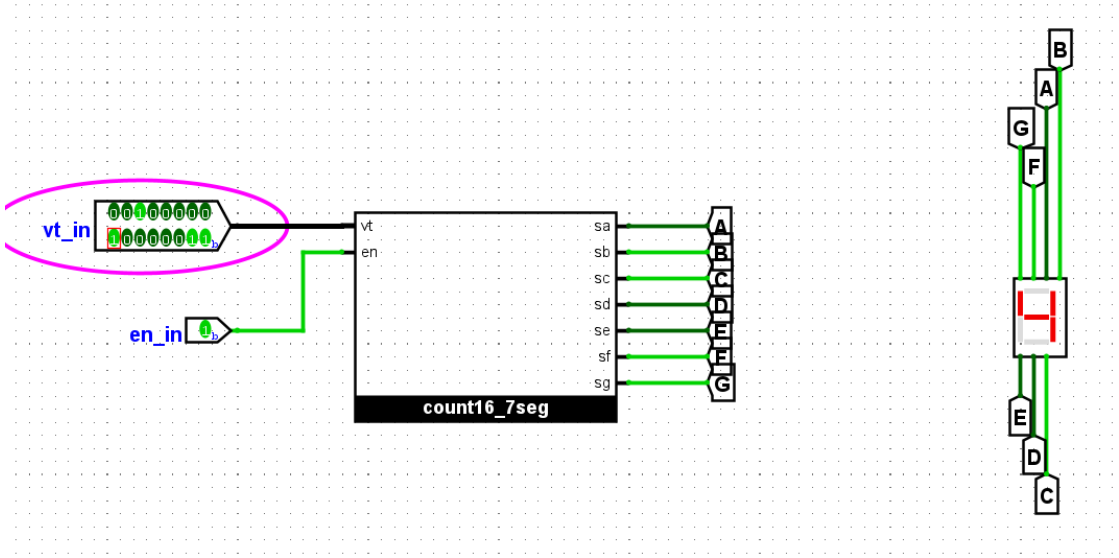


Figure 6. Count = 4, en = 1; observed display: 4.

4.6 Case 6 Count = 5

vt = 10100000 10000011; en = 1; pattern A–G = 1011011.

1 의 개수는 5 개이며, 관측한 표시 5 가 예상 결과와 일치합니다.

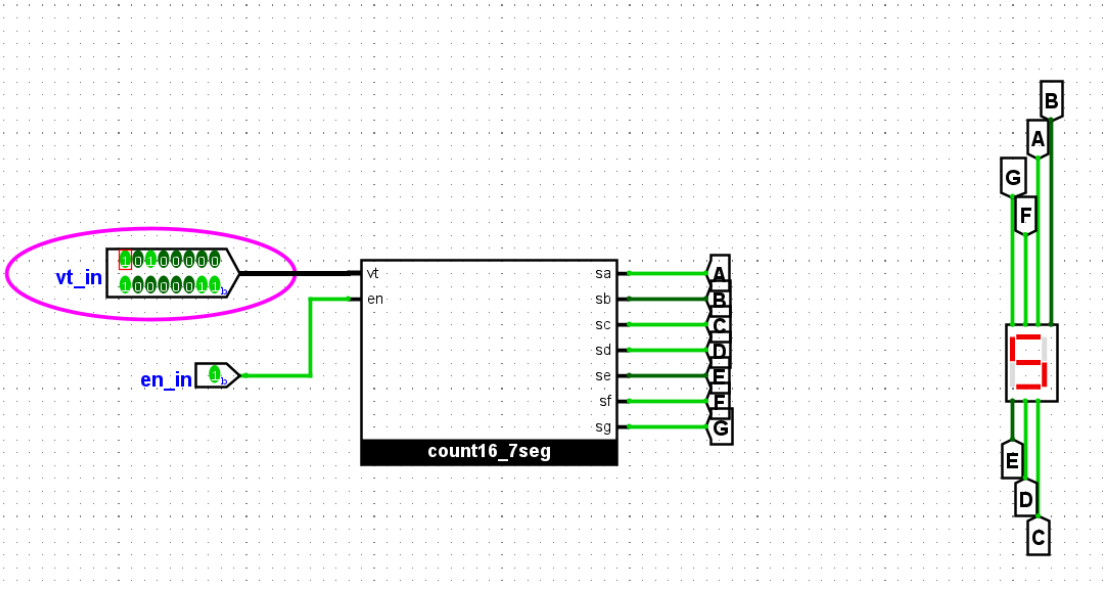


Figure 7. Count = 5, en = 1; observed display: 5.

4.7 Case 7 Count = 6

vt = 10100000 10001011; en = 1; pattern A–G = 1011111.

1 의 개수는 6 개이며, 관측한 표시 6 가 예상 결과와 일치합니다.

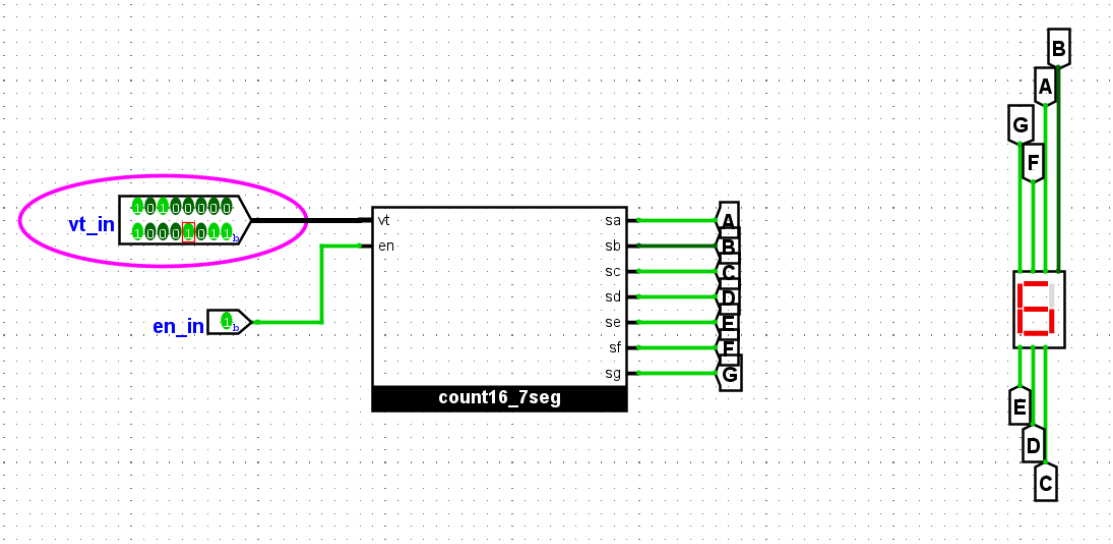


Figure 8. Count = 6, en = 1; observed display: 6.

4.8 Case 8 Count = 7

vt = 10100100 10001011; en = 1; pattern A–G = 1110000.

1 의 개수는 7 개이며, 관측한 표시 7 가 예상 결과와 일치합니다.

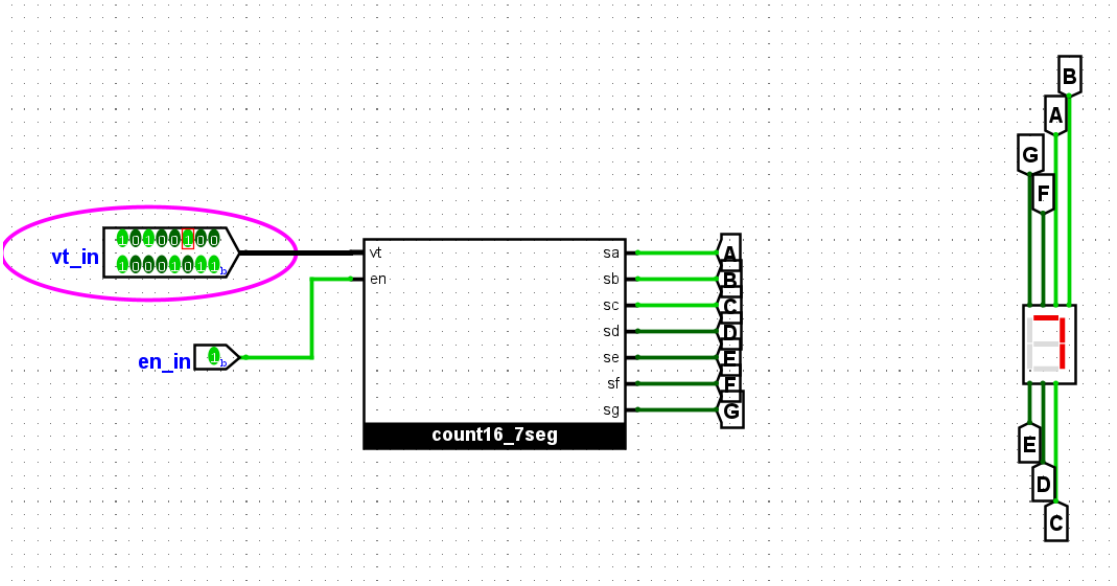


Figure 9. Count = 7, en = 1; observed display: 7.

4.9 Case 9 Count = 8

vt = 11100100 10001011; en = 1; pattern A–G = 1111111.

1 의 개수는 8 개이며, 관측한 표시 8 가 예상 결과와 일치합니다.

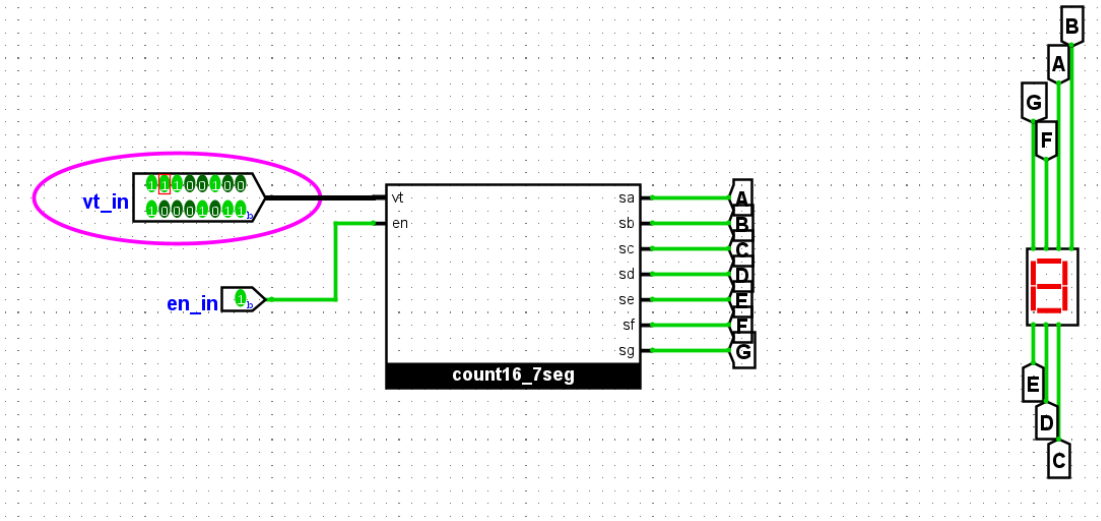


Figure 10. Count = 8, en = 1; observed display: 8.

4.10 Case 10 Count = 9

vt = 11100101 10001011; en = 1; pattern A–G = 1111011.

1 의 개수는 9 개이며, 관측한 표시 9 가 예상 결과와 일치합니다.

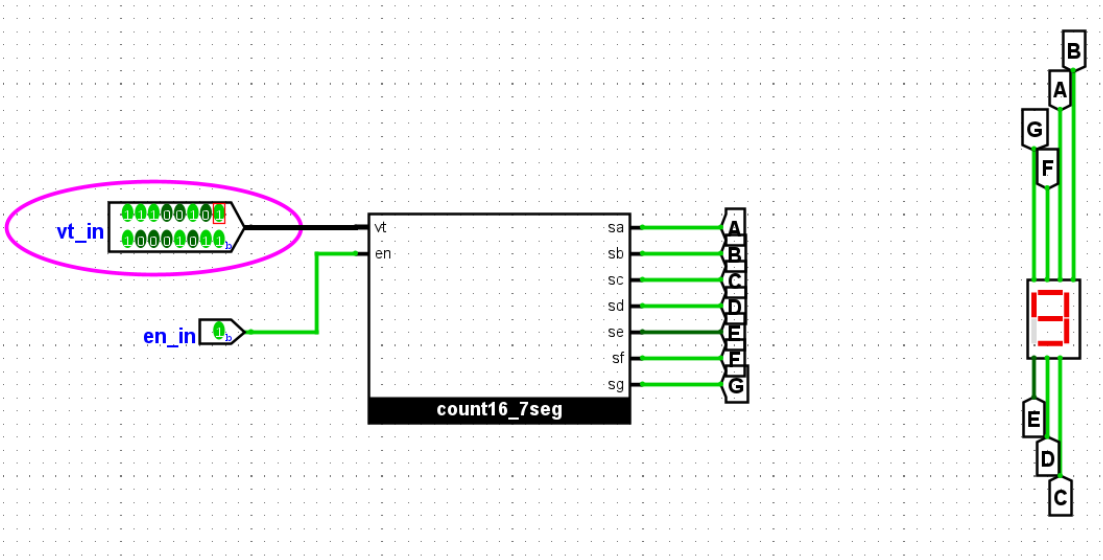


Figure 11. Count = 9, en = 1; observed display: 9.

4.11 Case 11 Count = 10

vt = 11100101 10101011; en = 1; pattern A–G = 1110111.

1 의 개수는 10 개이며, 관측한 표시 A 가 예상 결과와 일치합니다.

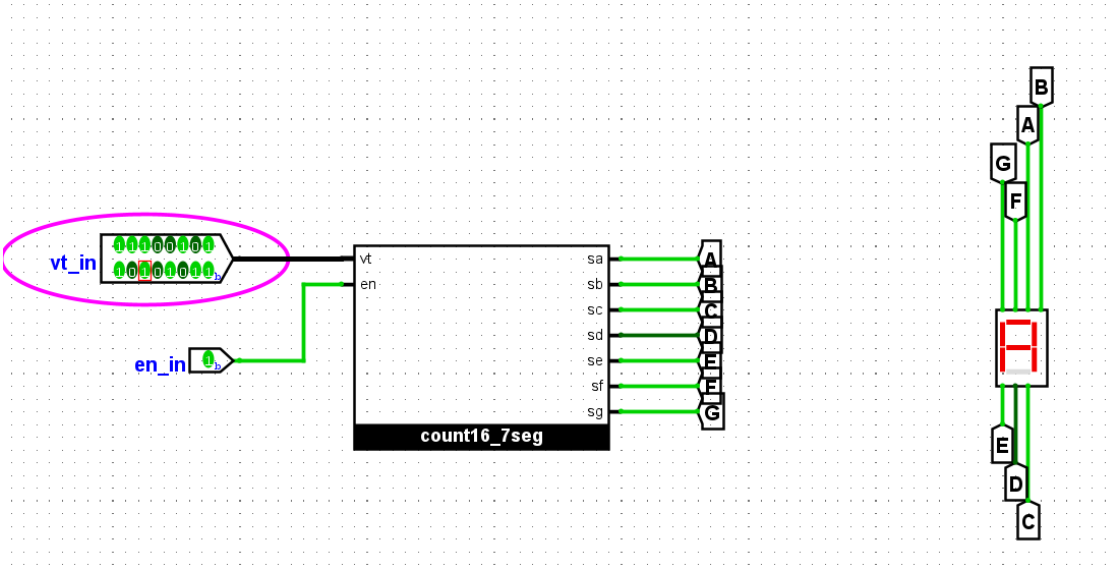


Figure 12. Count = 10, en = 1; observed display: A.

4.12 Case 12 Count = 11

vt = 11100101 11101011; en = 1; pattern A–G = 0011111.

1 의 개수는 11 개이며, 관측한 표시 b 가 예상 결과와 일치합니다.

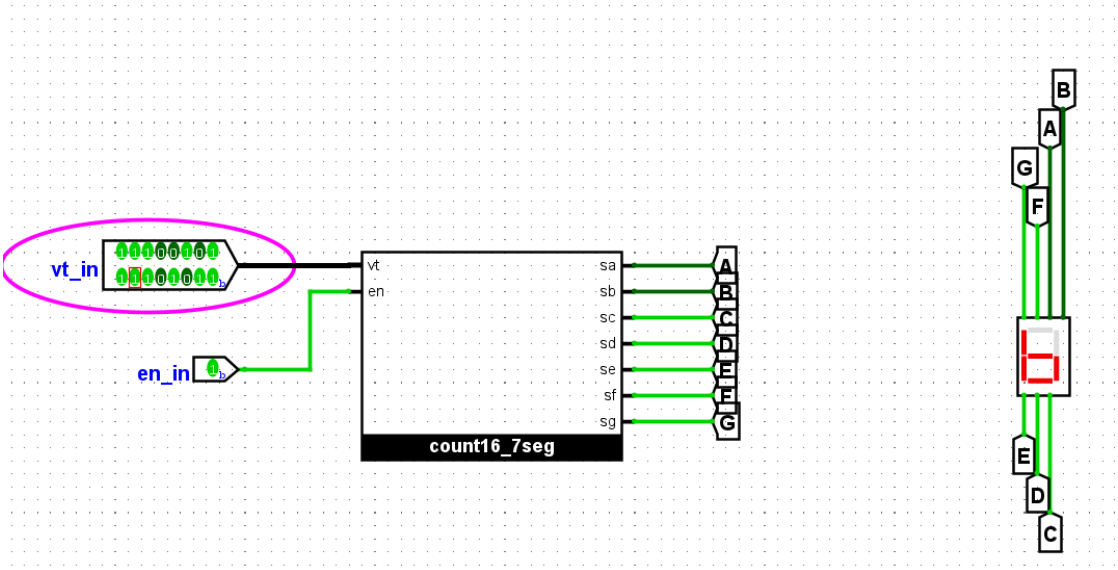


Figure 13. Count = 11, en = 1; observed display: b.

4.13 Case 13 Count = 12

vt = 11100101 11111011; en = 1; pattern A–G = 1001110.

1 의 개수는 12 개이며, 관측한 표시 C 가 예상 결과와 일치합니다.

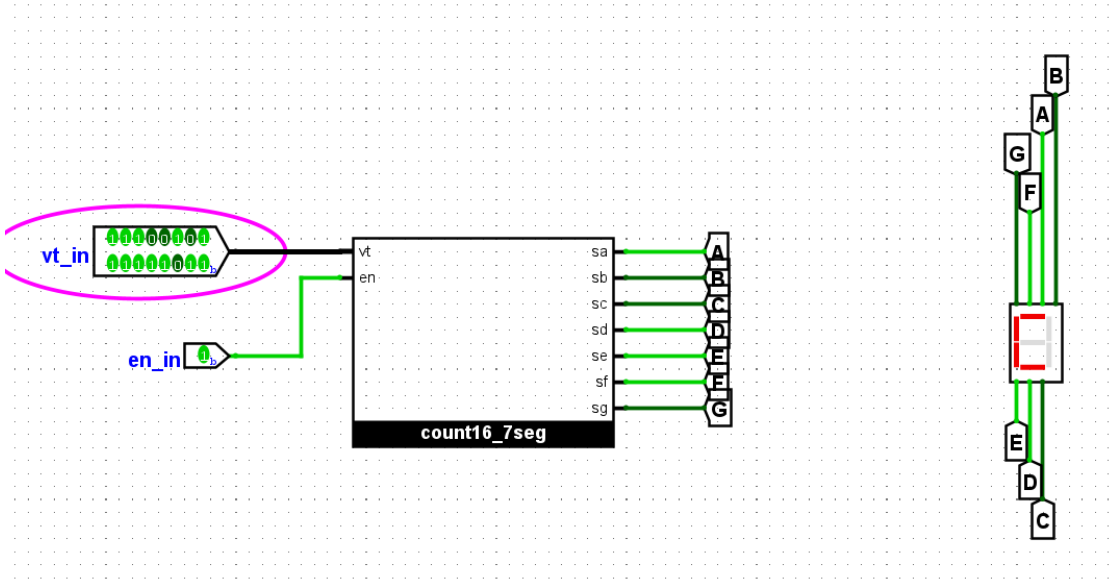


Figure 14. Count = 12, en = 1; observed display: C.

4.14 Case 14 Count = 13

vt = 11110101 11111011; en = 1; pattern A–G = 0111101.

1 의 개수는 13 개이며, 관측한 표시 d 가 예상 결과와 일치합니다.

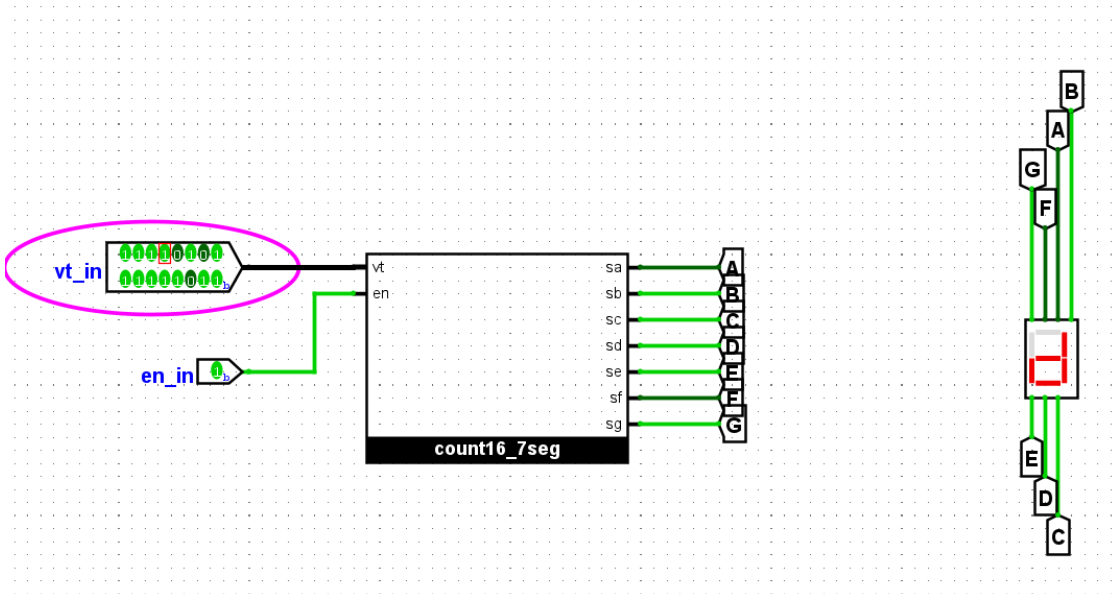


Figure 15. Count = 13, en = 1; observed display: d.

4.15 Case 15 Count = 14

vt = 11110101 11111111; en = 1; pattern A–G = 1001111.

1 의 개수는 14 개이며, 관측한 표시 E 가 예상 결과와 일치합니다.

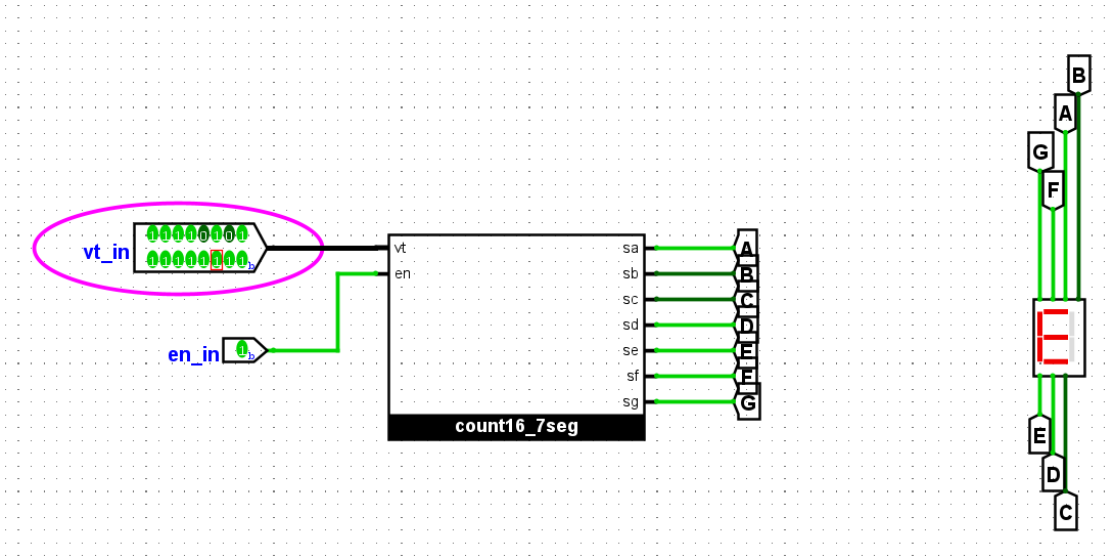


Figure 16. Count = 14, en = 1; observed display: E.

4.16 Case 16 Count = 15

vt = 11110111 11111111; en = 1; pattern A–G = 1000111.

1 의 개수는 15 개이며, 관측한 표시 F 가 예상 결과와 일치합니다.

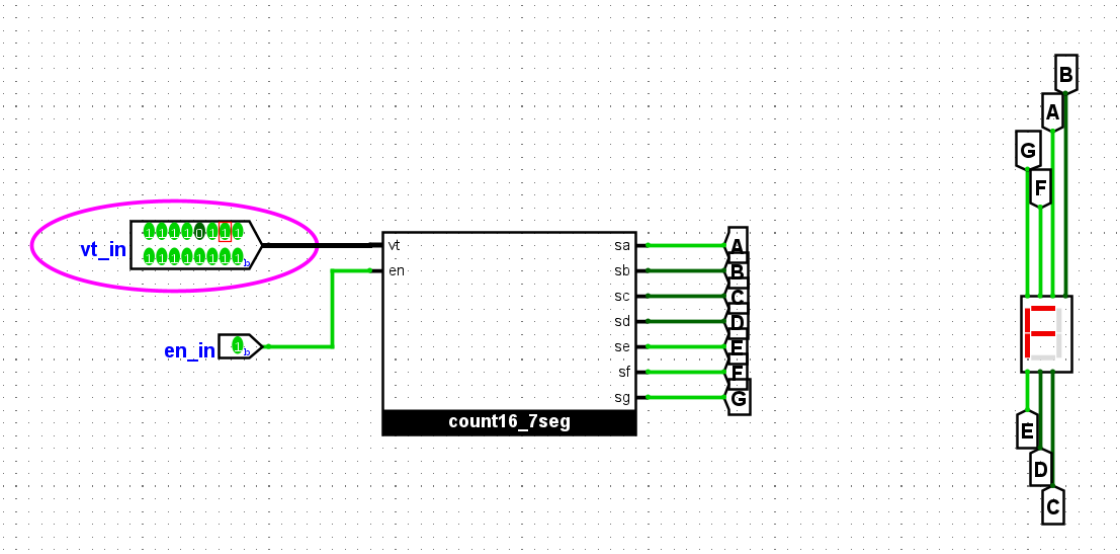


Figure 17. Count = 15, en = 1; observed display: F.

4.17 Case 17 Count = 16

vt = 11111111 11111111; en = 1; pattern A–G = 0000000.

1 이 16 개이므로 when others 에 따라 전체 소등합니다. 표시 범위를 초과한 경우에 대한 추가 설계 동작과 일치합니다.

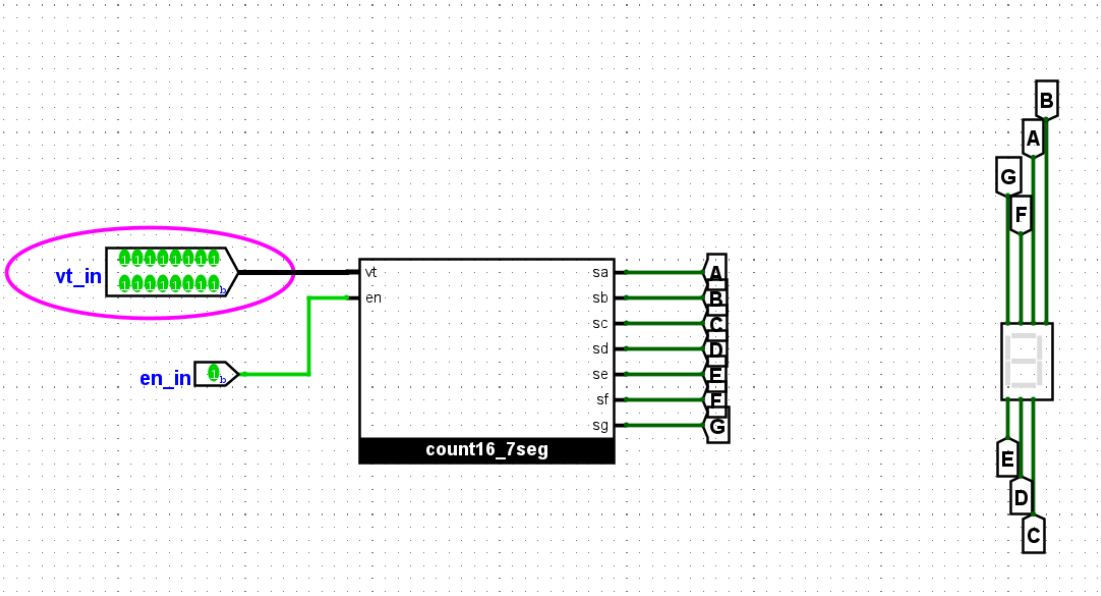


Figure 18. Count = 16, en = 1; observed display: Blank.

5. Conclusion

The VHDL module counts the ones in a 16-bit input and displays counts 0 to 15 as 0 to F. The screenshots verify all 17 possible count values, including the intentional blank display at 16. The enable-off branch is defined in the source code, but an en = 0 screenshot was not provided. This design therefore documents the required numeric and letter mapping together with an explicit policy for the all-ones input.

VHDL 모듈로 16 비트 입력의 1 의 개수를 세어 0~15 를 0~F 로 표시했습니다. 17 개 스크린샷에서 가능한 개수 0~16 을 확인했으며, 16 일 때 의도한 전체 소등도 확인했습니다. en = 0 의 소등 분기는 코드에 정의되어 있지만 해당 캡처는 제공되지 않았습니다. 이 설계는 요구된 숫자·문자 표시와 함께 모든 입력이 1 일 때의 처리 방침을 명확히 정의합니다.